TP2 – Product Flavors (Partie B)

Développement Mobile – BTS

Objectif de la partie B

Dans cette partie, l'objectif est de :

- conserver la **MainActivity** comme interface Étudiant
- créer une interface spécifique pour l'**Admin**
- rediriger l'utilisateur vers la bonne Activity après le login

1 1. Création de l'AdminActivity

Dans cette partie, une seule nouvelle Activity sera créée :

`AdminActivity`

Étapes

- Clic droit sur le package de l'application
- New → Activity → Empty Views Activity
- Nom : `AdminActivity`

Android Studio génère automatiquement :

- `AdminActivity.java`
- `activity_admin.xml`

2 2. Interface graphique Admin (XML)

Dans le fichier `activity_admin.xml`, écrire le layout suivant :

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:id="@+id/tvAdminTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Interface Admin"
        android:textSize="22sp"
        android:textStyle="bold" />
```

```
    <TextView
        android:id="@+id/tvAdminInfo"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Bienvenue dans l'espace administrateur" />

</LinearLayout>
```

FIGURE 1 – Interface Admin affichée

3 3. Code Java de l'AdminActivity

Dans le fichier `AdminActivity.java`, écrire le code suivant :

```
public class AdminActivity extends AppCompatActivity {

    private static final String LOG_TAG =
            AdminActivity.class.getSimpleName();

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_admin);

        Log.d(LOG_TAG, "AdminActivity ouverte");
    }
}
```

Explication :

- `AdminActivity` hérite de `AppCompatActivity`
- Le layout `activity_admin.xml` est chargé
- Le message dans Logcat confirme l'ouverture de l'interface Admin

4 4. Redirection après le login

Dans `LoginActivity.java`, utiliser le code suivant :

```
if (username.equals("admin") && password.equals("1234")) {

    Log.d(LOG_TAG, "Login réussi");

    if (BuildConfig.FLAVOR.equals("student")) {

        Log.d(LOG_TAG, "Mode STUDENT actif");
        Intent intent =
            new Intent(LoginActivity.this, MainActivity.class);
        intent.putExtra("username", username);
        startActivity(intent);

    } else {

        Log.d(LOG_TAG, "Mode ADMIN actif");
        Intent intent =
            new Intent(LoginActivity.this, AdminActivity.class);
        intent.putExtra("username", username);
        startActivity(intent);
    }

} else {
    Log.d(LOG_TAG, "Login échoué");
    Toast.makeText(this,
        "Wrong credentials ",
        Toast.LENGTH_SHORT).show();
}
```

5 5. Tests à effectuer

- Lancer `studentDebug` : ouverture de `MainActivity` (Étudiant)
- Lancer `adminDebug` : ouverture de `AdminActivity`

6 Conclusion de la partie B

Cette partie a permis de :

- créer une interface Admin distincte
- utiliser les Product Flavors pour la navigation
- comprendre un cas réel de gestion de rôles

A Annexe – Exemple avancé de AdminActivity

Cette annexe présente un exemple **optionnel** de `AdminActivity` plus riche.
Important :

- Ce code n'est **pas obligatoire** pour la séance
- Il sert uniquement de **référence** ou d'extension
- Il introduit des composants d'interface non encore étudiés

Fonctionnalités de l'interface Admin

- `Switch` : activer ou désactiver un mode maintenance
- `Spinner` : choisir un rôle utilisateur
- `Button` : appliquer les paramètres

Code Java – AdminActivity (Annexe)

```
public class AdminActivity extends AppCompatActivity {

    Switch switchMaintenance;
    Spinner spinnerRole;
    Button btnApply;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_admin);

        switchMaintenance = findViewById(R.id.switchMaintenance);
        spinnerRole = findViewById(R.id.spinnerRole);
        btnApply = findViewById(R.id.btnApply);

        // Données du Spinner
        String[] roles = {"Lecteur", "Modérateur", "Admin"};
        ArrayAdapter<String> adapter = new ArrayAdapter<>(
                this,
                android.R.layout.simple_spinner_dropdown_item,
                roles
        );
        spinnerRole.setAdapter(adapter);

        btnApply.setOnClickListener(v -> {

            String roleChoisi = spinnerRole.getSelectedItem().toString();
            boolean maintenance = switchMaintenance.isChecked();

            String message = "Rôle : " + roleChoisi +
                    " | Maintenance : " + (maintenance ? "ON" : "OFF");

            Toast.makeText(this, message, Toast.LENGTH_SHORT).show();
        });
    }
}
```

Remarque pédagogique

Cet exemple permet d'introduire progressivement :

— des composants interactifs avancés

— la récupération des valeurs depuis l'interface graphique

— une logique métier plus riche côté Admin

Il pourra être exploité dans une **séance ultérieure**.